Структури от данни и алгоритми. Анализ на алгоритми. Абстрактни типове от данни. Стек, опашка, списък, дърво. Сортиране.

1. **Основи на анализа на алгоритми: Асимптотична нотация на сложността. Основни рекурентни формули. Примери за анализ на алгоритми.**

Бързодействието на даден алгоритъм може да зависи от скоростта на даденият компютър, върху който се изпълнява. Различаваме относително бързодействие (върху даден компютър) и абсолютно бързодействие (върху различни компютри). При формалното оценяване на сложността на алгоритмите се интересуваме от поведението им при N (размер на входа), клонящо към безкрайност, като игнорираме машинно зависимите константи, тоест ще ни интересува как се държи алгоритъма при достатъчно голям размер N на входните данни, без да ни интересува машината, на която се изпълнява алгоритъма. Сложността на алгоритъм ще измерваме спрямо времето или паметта, необходима на алгоритъма за да реши даден проблем спрямо размера на подаденият вход (съответно сложност по време и сложност по памет).

Разпознаваме три вида анализ:

- Най-лош случай: (обикновено)
  T(n) = максималното време за алгоритъм върху всеки вход с размер n.

- Средно време: (понякога)
  T(n) = очакваното време за алгоритъм за всеки вход с размер n.
  Необходимо е предположение за статистическото разпределение на входа.

- Най-добър случай: (погрешно)
  Измама с бавен алгоритъм, който работи бързо върху някакъв вход.

За анализа на алгоритми ще използваме следните 'асимптотична нотации':

- **O-нотация (горна граница):**

Дефиниция: $O(g(n))= \{ f(n)$: за които съществуват константи $c> 0, n_0> 0$, такива че $0 \le f(n) \le cg(n)$ за всяко $n \ge n_0\}$

Записваме $f(n) = O(g(n))$ ако съществуват константи $c> 0, n_0> 0$, такива че $0 \le f(n) \le cg(n)$ за всяко $n \ge n_0$.

Пример: $2n^2= O(n^3)$:($c= 1, n_0= 2$)

- **Ω-нотация (долна граница)**

Дефиниция: $\Omega(g(n))= \{ f(n)$: за които съществуват константи $c > 0, n_0> 0$, такива че $0 \le cg(n) \le f(n)$ за всяко $n \ge n_0\}$)

Записваме $f(n) = \Omega(g(n))$ ако съществуват константи $c> 0, n_0> 0$, такива че $0 \le cg(n) \le f(n)$ за всяко $n \ge n_0$.

Пример: n $\in \Omega(\lg n)$ ($c=1, n_0= 16$)

- **Θ-нотация (tight bounds)**

Дефиниция: : $\Theta(g(n))= \{ f(n)$: за които съществуват константи $c_1 > 0, c_2> 0, n_0> 0$, такива че $0 \le c_1g(n) \le f(n) \le c_2g(n)$ за всяко $n \ge n_0\}$)

От дефиницията следва, че $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$.

Пример: 12n2 - $2n \in \Theta(n2)$

- **o-нотация**

Дефиниция: $o(g(n))= \{ f(n)$: ако съществуват константи $c> 0$, $n_0> 0$, такива че $0 \leq f(n) < cg(n)$ за всяко $n \geq n_0\}$

- **ω-нотация**

Дефиниция: $\omega(g(n))= \{ f(n)$: ако съществуват константи $c> 0$, $n_0> 0$, такива че $0 \leq cg(n) < f(n)$ за всяко $n \geq n_0\}$)

O-нотацията и Ω-нотацията са като ≤ и ≥.
o- нотацията и ω- нотацията са като < и >.

Основни рекурентни формули

Формула 1. Тази рекурентна формула възниква при рекурсивни програми, която изпълнява цикъл по входните данни за да елиминира една от тях:

$C_N = C_{N-1} + N$, за $N >= 2$ и $C_1 = 1$.

Решение: C_N е приблизително $N^2/2$.

$C_N = C_{N-1} + N$

$= C_{N-2} + (N-1) + N$

$= C_{N-3} + (N-2) + (N-1) + N$

…

Продължавайки по този начин, получаваме

$C_N = C_1 + 2 + \ldots + (N-2) + (N-1) + N$

$= 1 + 2 + \cdots + (N - 2) + (N - 1) + N$

$= N \,.\, (N+1) / 2.$

Формула 2. Тази формула се получава при рекурсивна програма, която разделя входа си на две половини на една стъпка:

$C_N = C_{N/2} + 1$, за $N >= 2$ и $C_1 = 1$.

Решение: C_N е приблизително lg N. Както е написано, това равенство е безсмислено, освен ако N не е четно и затова приемаме, че N/2 целочислено деление. Нека приемем за момент, че $N = 2^n$, така че рекурентната формула винаги да е добре дефинирана ($n = \lg N$.) Тогава:

$$
\begin{aligned}
C_{2^n} &= C_{2^{n-1}} + 1 \\
&= C_{2^{n-2}} + 1 + 1 \\
&= C_{2^{n-3}} + 3 \\
&\vdots \\
&= C_{2^0} + n \\
&= n + 1.
\end{aligned}
$$

Тоест C_N е приблизително lg N.

Формула 3. Тази рекурентна формула възниква при рекурсивни програми, които разделят входа на две половини, но освен това може би трябва да изследват всеки елемент от входа:

$C_N = C_{N/2} + N$, за $N >= 2$ и $C_1 = 0$.

Решение: C_N е приблизително 2N. Рекурентната формула се развива до сумата N + N/2 + N/4 + N/8 + ... Ако редицата е безкрайна, тази геометрична прогресия дава точно 2N. Тъй като използваме целочислено деление и спираме при 1, тази стойност е приближение на точния отговор.

Формула 4. Тази рекурентна формула възниква при рекурсивни програми, които трябва да направят линейно обхождане на входа преди, по време или след разделянето на входа на две половини::

$C_N = 2C_{N/2} + N$, за $N >= 2$ и $C_1 = 0$.

Решение: C_N е приблизително N lg N. Това решение е най-широко цитирано, защото рекурентната формула е приложима за семейство стандартни алгоритми разделяй-и-владей.

$$
\begin{aligned}
C_{2^n} &= 2C_{2^{n-1}} + 2^n \\
\frac{C_{2^n}}{2^n} &= \frac{C_{2^{n-1}}}{2^{n-1}} + 1 \\
&= \frac{C_{2^{n-2}}}{2^{n-2}} + 1 + 1 \\
&\vdots \\
&= n.
\end{aligned}
$$

От тук следва, че $C_N \approx N \lg N$

Формула 5. Тази рекурентна формула възниква при рекурсивни програми, които разделят входа на две половини и освен това правят константна друга обработка.

$C_N = 2C_{N/2} + 1$, за $N >= 2$ и $C_1 = 1$.

Решение: C_N е приблизително 2N. Можем да изведем това решение по същия начин, както при формула **4**.

Примери за анализ на алгоритми

Пример 1: Последователно търсене

Тази функция проверява дали числото v е сред числата a[l], a[l+1], ..., a[r], като прави сравнения с всяко число започвайки отначалото. Ако достигнем края и не намерим числото, връщаме -1, иначе връщаме индекса на елемента от масива, съдържащ числото.

```
static int search(int a[], int v, int l, int r)
  { int i;
    for (i = l; i <= r; i++)
      if (v == a[i]) return i;
    return -1;
  }
```

Свойство 1

Последователното търсене изследва N числа при всяко неуспешно търсене и приблизително N/2 числа средно за всяко успешно търсене..

Ако всяко число в таблицата е равно вероятно да бъде търсено, тогава

(1 + 2 + … + N-1 + N)/N = (N + 1)/2

е средната цена на успешно търсене.

От свойство 1 следва, че времето за изпълнение на Пример 1 е пропорционално на N, ако приемем, че средната цена за сравняване на две числа е константа.

Можем да ускорим последователното търсене (особено при неуспешно търсене), ако сортираме числата в таблицата. В подредена таблица можем да прекратим търсенето веднага след достигането на число по-голямо от това, което търсим.

Свойство 2

Последователното търсене в подредена таблица изследва N числа в най-лошия случай и приблизително N/2 числа средно.

Пример 2: Двоично търсене

Тази програма има същата функционалност като Пример 1, но е по-ефективна.

```
static int search(int a[], int v, int l, int r)
  {
    while (r >= l)
      { int m = (l+r)/2;
        if (v == a[m]) return m;
        if (v < a[m]) r = m-1; else l = m+1;
      }
    return -1;
  }
```

Пример 2 е класическо решение на задачата за търсене, което е много по-ефективно от последователното търсене. Тя е основана на идеята, че ако числата в таблицата са подредени, можем да елиминираме половината като сравним търсеното число с числото в средата на таблицата. Ако числата са равни, имаме успешно търсене. Ако търсеното число е по-малко, прилагаме същия метод върху лявата половина на таблицата. Ако е по-голямо – прилагаме същия метод върху дясната половина на таблицата.

Свойство 3

Двоичното търсене никога не изследва повече от $\lfloor \lg N \rfloor + 1$ *числа.*

Доказателство: Нека T_N представя броя на сравненията в най-лошия случай. Тогава от начина, по който алгоритъмът редуцира търсенето в таблица с размер N в търсене в таблица с половината размер следва:

$$T_N \leq T_{\lfloor N/2 \rfloor} + 1, \quad \text{for } N \geq 2 \text{ with } T_1 = 1.$$

От тук като приложим разсъжденията като при Формула 2, ще получим, че

$T_N <= n + 1$ ако $N = 2^n$, което доказва твърдението.

2. Абстрактни типове от данни. Интерфейс и реализация.

Абстрактните типове данни позволяват да се пишат програми, които използват високо ниво на абстракция. Чрез тях може да се разделят концептуалните трансформации върху данните от конкретното представяне на структурите от данни и реализацията на

алгоритмите. Всички компютърни системи са основани на нива на абстракция като например:

- Абстрактен модел на бит с двоична стойност
- Абстрактен модел на компютъра
- Абстрактен модел на език за програмиране
- Абстрактна идея за алгоритъм, реализиран чрез програма на Java
- Абстрактни типове данни – позволяват да се разработят абстрактни механизми на по-високо ниво

За да се разработи ново ниво на абстракция е необходимо:

- *Да дефинираме* абстрактните обекти и операциите върху тях;
- *Да представим* данните чрез определени структури от данни;
- *Да реализираме* операциите.

Необходимо е също *да се отдели* клиента от реализацията по такъв начин, че един клиент да може да ползва множество реализации и една реализация да може да се ползва от много лиенти без да трябва да се променя кода.

Дефиниция. Абстрактен тип данни (АТД) е тип данни (множество от допустими стойности и операции над тях), който е достъпен само чрез **интерфейс**. Програма, която използва АТД се
нарича **клиент**, а програма, която определя типа данни се нарича **реализация**.
Пример: в Java – Java virtual machine е ниво на абстракция, което отделя Java програмите от реализацията на виртуалната машина.
При АТД клиентските програми нямат достъп до данните освен чрез операциите, зададени в интерфейса. Представянето на данните и реализацията на операциите са напълно отделено от клиента чрез интерфейса. Клиентът не може да види реализацията чрез интерфейса.
Пример: Реализация на клас **Point** - Дефинира тип данни, който се състои от множество от двойки реални числа (представящи точки в равнината). Представянето на данните е private. Класът Включва осем метода:

- 2 конструктора
- 2 метода за достъп
- 2 метода за преобразуване в полярни координати
- Метод за изчисляване разстояние между точки
- Метод toString

Реализация:

```
class Point
  {
    private double x, y;
    Point()
      { x = Math.random(); y = Math.random(); }
    Point(double x, double y)
      { this.x = x; this.y = y; }
    double x() { return x; }
    double y() { return y; }
```

```
  double r() { return Math.sqrt(x*x + y*y); }
  double theta() { return Math.atan2(y, x); }
  double distance(Point p)
    { double dx = this.x() - p.x();
      double dy = this.y() - p.y();
      return Math.sqrt(dx*dx + dy*dy);
    }
  public String toString()
    { return "(" + x + ","+y+")"; }
}
```

Клиентските програми не могат да използват полетата x и y, защото са private. Могат да използват само публичните методи.
Не позволявайки клиентските програми да достъпват директно данните, ние сме свободни да променим представянето. Ще разгледаме друга реализация на клас **Point** – вътрешното представяне на точките е чрез полярни координати:

```
class Point
 {
  private double r, theta;
  Point()
    { double x = Math.random(), y = Math.random();
      this = new Point(x, y); }
  Point(double x, double y)
    { r = Math.sqrt(x*x + y*y);
      theta = Math.atan2(y, x); }
  double r() { return r; }
  double theta() { return theta; }
  double x() { return r*Math.cos(theta); }
  double y() { return r*Math.sin(theta); }
  double distance(Point p)
    { double dx = x() - p.x();
      double dy = y() - p.y();
      return Math.sqrt(dx*dx + dy*dy);
    }
  public String toString()
    { return "(" + x() + ", " + y() + ")"; }
}
```

Ще извеждем дефиницията на интерфейса от реализацията на класа като изтрием private членовете, реализацията на public методите и имената на параметрите, като оставим само прототипите на public методите:

```
class Point // ADT interface
 { // implementations and private members hidden
  Point()
  Point(double, double)
  double x()
  double y()
  double r()
  double theta()
```

```
  double distance(Point)
  public String toString()
}
```

Дефинирайки и използвайки интерфейси ние можем да използваме различни реализации, които имат един и същи интерфейс, без да променяме кодът във клиентската програма, която използва АТД. В горният пример ние можем да използваме и двете реализации на класа Point без да променяме клиентската програма. Дефинициите на интерфейсите служат за кратко и пълно описание на договора между клиентите и реализациите, който е основата на ефективното проектиране на АТД.

3. **Свързани списъци. Обработка на списъци.**

Дефиниция: Свързан списък е множество от елементи (items), където всеки елемент е част от **възел** (**node**), който също съдържа **връзка** (**link**) към възел.
Когато основният ни интерес е да обходим последователно съвкупност от елементи, можем да ги организираме в *свързан списък* – основна структура от данни, където всеки елемент съдържа информация как да стигнем до следващия елемент. Това ни дава преимущество за ефективна реорганизация на елементите на свързаният списък, за сметка на липсата на бърз достъп до произволен елемент на списъка - необходимо е последователно обхождане на елементите на списъка докато се достигне желаният елемент. Достъпът до началото на свързания списък е пряк. Има два основни начина за представяне на свързания списък в паметта: с една или с две връзки – съответно едносвързан и двусвързан списък. При едносвързаният списък елементите имат достъп само до съседа си от едната страна, докато при двусвързаният списък елементите имат достъп и до двамата си съседи.
За последния възел имаме следните конвенции:

- Той е нулева връзка (null link), която не сочи никъде
- Реферира се фалшив възел (dummy node), който не съдържа елемент (item).
- Реферира се назад към първия възел, като по този начин списъкът става цикличен

Реализация на Java :

```
class Node
{ Object item; Node next; }
```

Алокиране на памет:

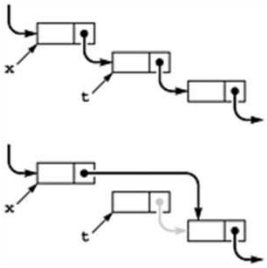
```
Node x = new Node();
```

Инициализация на елементите:

```
Class Node
{ Object item; Node next;
  Node(Object v)
  { item = v; next = null; }
}
...
t = new Node(x);
```

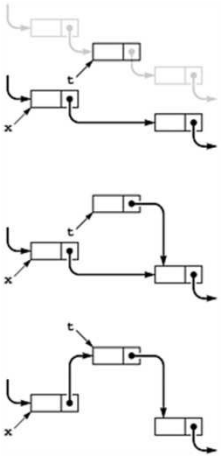
При свързаният списък операциите включване и изключване са допустими на произволно място:

- Изтриване на възел



```
t = x.next; x.next = t.next;    или   x.next = x.next.next;
```

- Вмъкване на възел



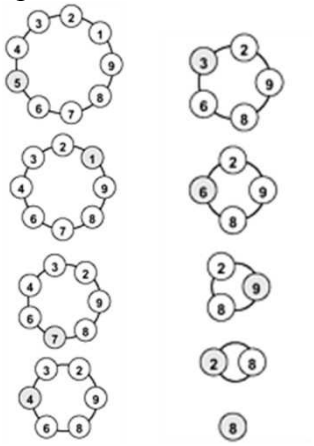
```
t.next = x.next; x.next = t;
```

Пример: Изборът на Josephus ***//този пример може да се изпусне за да бъде по-кратка темата?***
Примерът се изразява в следното: Искаме да изберем един човек от група хора за лидер. За целта хората се нареждат в кръг. Обхождаме кръга с хора и елиминираме всеки M-ти човек (M е дадено число), докато не остане само един човек.
Дадената ситуация може лесно да се моделира със цикличен свързан списък, където всеки елемент представлява човек и има достъп до следващият елемент от списъка.
Реализация на Изборът на Josephus:
Нека N е броя на хората, а M е поредният човек, който ще бъде отстранен:

```
class Josephus
  {
   static class Node
     { int val; Node next;
       Node(int v) { val = v; }
     }
   public static void main(String[] args)
     { int N = Integer.parseInt(args[0]);
       int M = Integer.parseInt(args[1]);
       Node t = new Node(1);
       Node x = t;
       for (int i = 2; i <= N; i++)
         x = (x.next = new Node(i));
       x.next = t;
```

```
    while (x != x.next)
      {
       for (int i = 1; i < M; i++) x = x.next;
       x.next = x.next.next;
      }
    Out.println("Survivor is " + x.val);
  }
}
```

При N = 9 и M = 5 изпълнението на примера изглежда по следният начин:



Ще разгледаме следните операции от обработката на списъци: обхождане на списък, обръщане на списък и сортиране на списък:

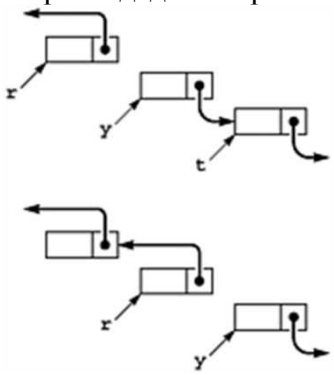
- обхождане на списък

  Ако приемем, че методът visit()извършва желано от нас действие на даден елемент, то можем да обходим елементите на свързан списък по следният начин:

```
for (Node t = x; t != null; t = t.next) visit(t.item);
```

- обръщане на списък

Можем да обърнем даден свързан списък по следният начин:



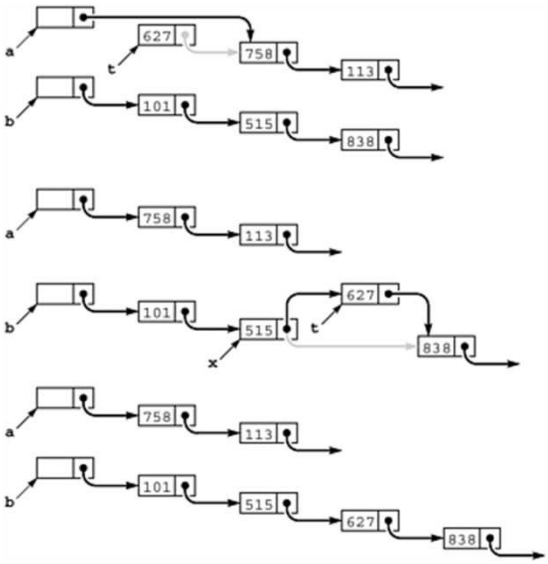
- ☐ r – указател към вече обработения списък
- ☐ y - указател към необработената част
- ☐ Запазваме указател към възела следващ y в t
- ☐ Променяме връзката на y да сочи r
- ☐ Преместваме r да сочи y, а y да сочи t

Реализация:

```
static Node reverse(Node x)
  { Node t, y = x, r = null;
    while (y != null)
      { t = y.next;
        y.next = r;
        r = y;
        y = t;
      }
    return r;
  }
```

- сортиране на списък

Сортирането се изразява в трансформирането на несортиран свързан списък (a) в сортиран (b), използвайки сортиране чрез вмъкване. Това, което правим е да обхождаме елементите на първият списък и да намираме правилното им място във втория, след което да ги вмъкваме, запазвайки вторият списък сортиран.



Реализация:

```
class ListSortExample
  {
    static class Node
      { int val; Node next;
        Node(int v, Node t) { val = v; next = t; }
      }
    static Node create()
      { Node a = new Node(0, null);
        for (In.init(); !In.empty(); )
          a.next = new Node(In.getInt(), a.next);
        return a;
      }
    static Node sort(Node a)
```

```
    { Node t, u, x, b = new Node(0, null);
      while (a.next != null)
        {
          t = a.next; u = t.next; a.next = u;
          for (x = b; x.next != null; x = x.next)
            if (x.next.val > t.val) break;
          t.next = x.next; x.next = t;
        }
      return b;
    }
  static void print(Node h)
    { for (Node t = h.next; t != null; t = t.next)
        Out.println(t.val + ""); }
  public static void main(String[] args)
    { print(sort(create())); }
}
```

4. **Структура от данни стек. Реализация.**

Дефиниция: Стек е АТД (абстрактен тип данни), който има две основни операции: вмъкване (push) на нов елемент и изтриване (pop) на елемента, който последно е вмъкнат.

Интерфейс на АТД стек:

```
class intStack // ADT interface
{ // implementations and private
// members hidden
intStack(int)
boolean isEmpty()
void push(int)
int pop()
}
```

Стекът е крайна редица от елементи от един и същи тип. Операциите включване и изключване на елементи от стека са позволени само в единият му край наречен ‘връх на стека’. Възможен е достъп само до елемента, намиращ се на върха на стека, като достъпът е пряк. При тази организация на логическите оперативи последният включен елемент се изключва пръв. Тоест стекът представлява структура от данни с поведение FILO – First In Last Out. Структурата от данни стек има две основни реализации– динамична и статична:

- **Статичен стек (реализация с масив)**

```
class intStack
{
  private int[] s;
  private int N;
  intStack(int maxN)
    { s = new int[maxN]; N = 0; }
  boolean isEmpty()
    { return (N == 0); }
  void push(int item)
    { s[N++] = item; }
  int pop()
    { return s[--N]; }
}
```

- **Свързан стек (динамична реализация)**

```
class intStack
{
  private Node head;
  private class Node
    {
      int item; Node next;
      Node(int item, Node next)
        { this.item = item; this.next = next; }
    }
  intStack(int maxN)
    { head = null; }
  boolean isEmpty()
    { return (head == null); }
  void push(int item)
    { head = new Node(item, head); }
  int pop()
    { int v = head.item; Node t = head.next;
      head = t; return v; }
}
```

5. **Структура от данни опашка. Реализация**

Дефиниция: FIFO опашка е АТД, който се състои от две основни операции: вмъкване (put) на нов елемент и премахване (get) на елемента, който е вмъкнат най-рано.
Интерфейс на АТД FIFO опашка:

```
class intQueue // ADT interface
{ // implementations and private
// members hidden
intQueue(int)
boolean isEmpty()
void put(int)
int get()
}
```

Опашката е крайна редица от елементи от един и същи тип. Операцията включване е допустима за елементите от единия край на редицата, който се нарича ‘край на опашката’, а операцията изключване на елемент ь от другия край на редицата, който се нарича ‘начало на опашката’. Възможен е достъп само до елемента , намиращ се в началото на опашката, като достъпът е пряк. При тази организация на логическите операции първият включен елемент се изключва пръв. Тоест опашката представлява структура от данни с поведение FIFO – First In First Out. Структурата от данни опашка има две основни реализации– динамична и статична:

- **Статична опашка (реализация с масив)**

```
class intQueue
{
  private int[] q; private int N, head, tail;
  intQueue(int maxN)
    { q = new int[maxN + 1];
      N = maxN + 1; head = N; tail = 0; }
  boolean empty()
    { return (head % N == tail); }
  void put(int item)
    { q[tail++] = item; tail = tail % N; }
  int get()
    { head = head % N; return q[head++]; }
```

```
}
```

- **Свързана опашка (динамична реализация)**

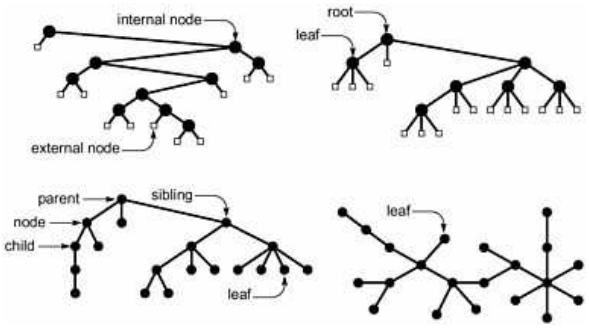
```
class intQueue
{
  private class Node
    { int item; Node next;
      Node(int item)
        { this.item = item; next = null; }
    }
  private Node head, tail;
  intQueue(int max)
    { head = null; tail = null; }
  boolean empty()
    { return (head == null); }
  void put(int item)
    { Node t = tail; tail = new Node(item);
      if (empty()) head = tail; else t.next = tail;
    }
  int get()
    { int v = head.item; Node t = head.next;
      head = t; return v; }
  }
```

6. **Дървета. Типове дървета**.

Дърветата са математическа абстракция,която играе централна роля при проектирането и анализа на алгоритми. Дърветата могат да бъдат използвани за да се да опишат динамичните
свойства на алгоритмите, както за построяването и използването на явни структури от данни, които са конкретни реализации на дървета. Много примери за дървета могат да бъдат намерени във всекидневният живот: семейни дървета, организацията на спортни турнири, структурата на големи организации, файловата система при компютрите и други.
Съществуват много различни типове дървета. Ще дефинираме дърветата като абстрактни обекти и ще въведем основната терминология. Ще разгледаме неформално различни типове дървета по ред на намаляване на тяхната общност:

- Дървета
- Дървета с корен (rooted trees)
- Подредени дървета
- М-арни дървета и двоични дървета

Двоично дърво 3-ично дърво



Дърво с корен Свободно дърво

- *Дърво* е не празна съвкупност от възли и връзки (ребра), които удовлетворяват определени изисквания.
- *Възел* (vertex, node) е прост обект, който може да има име и да носи асоциирана информация.
- *Ребро* (edge) е връзка между два възела.
- *Път* в дърво е списък от различни възли, в който последователните възли са свързани чрез ребра в дървото.
- Дефиниращо свойство на дърво: съществува точно един път свързващ всеки два възела в дървото.
- Ако съществува повече от един път между някои възли или не съществува път между някои възли, тогава имаме граф, а не дърво.
- Непресичащо се множество от дървета се нарича *гора*.
- *Дърво с корен* е такова дърво, в което сме избрали един възел за *корен*.
- **Дефиниция:** *Дърво с корен* (или неподредено дърво) е възел (наречен корен) свързан с множество от дървета с корен.
- **Дефиниция:** *Дърво* (наричано още подредено дърво) е възел (наречен *корен*), който е свързан с редица от несвързани дървета. Такава редица се нарича *гора*.
- Съществува точно един път между корена и всеки друг възел в дървото.
- Казваме, че възел у е *под* възел х (и х е над у), ако х е на пътя от у до корена.
- Всеки възел (без корена) има точно един възел над него, който се нарича негов *родител*. Възлите директно под даден възел се наричат негови *деца*.
- Възли без деца се наричат *листа* или *терминални* възли. Възли, които имат поне едно дете се наричат *нетерминални* възли.
- *Подредено дърво* е дърво с корен, при което е указана подредбата на децата на всеки
  възел
- Ако всеки възел *трябва* да има определен брой деца, подредени по определен начин, тогава имаме *М-арно дърво*.
- При такова дърво, често е подходящо да дефинираме специални външни възли, които нямат деца.
- **Дефиниция:** *М-арно дърво* е или външен възел, или вътрешен възел, свързан с подредена редица от М дървета, които също са М-арни дървета.
- Листо в М-арно дърво е вътрешен възел, всички деца на който са външни възли.

- Разликата между подредените и M-арните дървета, е че възлите в подредените дървета могат да имат произволен брой възли, докато възлите на M-арните дървета имат точно M деца.
- В частност, най-простият тип M-арно дърво е двоичното дърво. *Двоично дърво* е подредено дърво, състоящо се от два типа възли: външни възли без деца и вътрешни възли с точно две деца. Тъй като двете деца на всеки вътрешен възел са подредени, ще ги наричаме *ляво дете* и *дясно дете*.
- **Дефиниция:** Двоично дърво е или външен възел, или вътрешен възел, свързан с двойка двоични дървета, които се наричат ляво поддърво и дясно поддърво на този възел.

7. **Сортиране. Елементарни методи за сортиране.**

Най-общо казано сортирането представлява процесът на пренареждане на елементите на някакво множество от обекти в определен ред. Сортирането е основна дейност с широка сфера на приложение – речници, телефонни указатели, справочни индекси и други – навсякъде, където се налага бързо търсене и намиране на различни обекти.

Ще разгледаме методи за сортиране на множества от обекти съдържащи "ключове". "Ключовете", които са обикновено малка част от обекта, се използват за да контролорат сортирането. Целта на един метод за сортиране е да пренареди обектите, така че техните ключове да са подредени по дадено добре определено правило – обикновено числова или азбучна подредба.

Съществуват различни класификации на на алгоритмите за сортиране. Една от най-популярните е в зависимост от местонахождението на данните: по този критерии различаваме вътрешно сортиране, когато данните се намират в оперативната памет на компютъра, и външно сортиране, когато данните са във външна памет за компютъра. Главната разлика между двете е , че при вътрешното сортиране можем да достъпваме обектите директно, докато при външното сортиране достъпът до данните става последователно, или на големи блокове, като се започва от първия елемент. Друга важна класификация на алгоритмите е според това дали те са "устойчиви" или "неустойчиви".

- Дефиниция: Един метод за сортиране се нарича "устойчив", ако запазва относителната подредба на елементите със равни ключове.

Например ако вземем сортиран по азбучен ред списък със ученици, съдържащ името на ученик и годината на завършване и сортираме този списък по година със устойчив метод, то учениците завършили в една и съща година ще са все още сортирани по азбучен ред в изходният списък. Ако използваме неустойчив метод за сортиране първоначалният относителен ред на учениците най-вероятно няма да се запази.

Ще разгледаме реализацията на три елементарни метода за сортиране:

- Сортиране чрез селекция (selection sort)
- Сортиране чрез вмъкване (insertion sort)
- Сортиране чрез метод на мехурчето (bubble sort)

И трите елементарни метода за сортиране отнемат време пропорционално на N^2 за да сортират N на брой произволно подредени елемента. Ако N е малко, то тези методи за сортиране са напълно достатъчни. Те дори могат да бъдат по-бързи от някои по-сложни методи за сортиране при малко N или в някои други случай. Но когато броят N на елементите е голям и те

са разпределени произволно, то гореспоменатите методи стават неефективни поради голямото време за изпълнение.
За представянето и реализацията на алгоритмите ще използваме методите:

- `less` – сравнява 2 елемента
- `exch` – разменя 2 елемента
- `compExch` – сравнява 2 елемента и ги разменя, ако е необходимо

- Сортиране чрез селекция (selection sort)
Сортирането чрез селекция работи по следният начин: Първо намираме наи-малкият елемент в масива и го разменяме със елемента на първа позиция. След това намираме вторият наи-малък елемент и го разменяме със елемента на втора позиция. Породължаваме по същият начин докато целият масив не е сортиран. Този метод се нарича сортиране чрез селекция, защото работи като постоянно се избира наи-малкият оставащ елемент.

Реализация на метода Сортиране чрез селекция:

```
static void selection(ITEM[] a, int l, int r)
  {
    for (int i = l; i < r; i++)
      { int min = i;
        for (int j = i+1; j <= r; j++)
          if (less(a[j], a[min])) min = j;
        exch(a, i, min);
      }
  }
```

- Сортиране чрез вмъкване (insertion sort)

Сортирането чрез вмъкване работи по следният начин: последователно обхождаме един по един елементите и ги поставяме на правилната им позиция. За вмъкването на всеки елемент на правилното му място се налага последователното му сравнение със вече сортираните елементи до откриване на вярната позиция и след това преместване на по-големите от него елементи с една позиция надясно.

Реализация на метода Сортиране чрез вмъкване:

```
static void insertion(ITEM[] a, int l, int r)
  { int i;
    for (i = r; i > l; i--) compExch(a, i-1, i);
    for (i = l+2; i <= r; i++)
      { int j = i; ITEM v = a[i];
        while (less(v, a[j-1]))
          { a[j] = a[j-1]; j--; }
        a[j] = v;
      }
  }
```

- Сортиране чрез метод на мехурчето:

Сортирането чрез вмъкване работи по следният начин: Нека N е броят на елементите, които ще се сортират. Последователно обхождаме елементите на структурата от дясно наляво N на бой пъти и ако има нужда разменяме съседните елементи. По този начин в процеса на сортиране най-малкият елементи, подобно на мехурчета изплуват към началото на масива, откъдето идва и името на метода.

Реализация на метода чрез метод на мехурчето:

```
static void bubble(ITEM[] a, int l, int r)
  { for (int i = l; i < r; i++)
      for (int j = r; j > i; j--)
        compExch(a, j-1, j);
 }
```

8. **Сортиране - QuickSort.**

Сортирането QuickSort е метод за сортиране от типа „Разделяй и владей". Алгритъмът работи като масива се разделя на две части, на базата на някакъв елемент, и след това се двете части се сортират независимо. Основната част от метода е разделянето, което пренарежда масива, така че следните три условия да са спазени:

- Елементът a[i]е на крайната си позиция в масива за някое i
- Никой от елементите a[l], ..., a[i-1] не е по-голям от a[i]
- Никой от елементите a[i+1], ..., a[r] не е по-малък от a[i]

Използваме следната стратегия за да реализираме разделянето: Първо произволно избираме a[i] да бъде разделящият елемент – този, който ще отиде на крайната си позиция. След това обхождаме, започвайки от левият край на масива докато намерим елемент по-голям от разделящият елемент. Също така обхождаме от десният край на масива докато не намерим елемент по-малък от разделящият елемент. Двата намерени елемента от обхождането очевидно не са на местата си в крайно наредениият масив, и затова ги разменяме. Продължавайки по същият начин сме сигурни, че никой елемент от ляво на левият индекс е по-голям от разделящият елемент и никой елемент от дясно на десният индекс е по-малък от разделящият елемент.

Реализация на QuickSort:

```
static void quicksort(ITEM[] a, int l, int r)
 {
  if (r <= l) return;
  int i = partition(a, l, r);
  quicksort(a, l, i-1);
  quicksort(a, i+1, r);
 }
```

Където реализацията на разделящият метод partition следва:

```
static int partition(ITEM a[], int l, int r)
 { int i = l-1, j = r; ITEM v = a[r];
  for (;;)
    {
     while (less(a[++i], v)) ;
     while (less(v, a[--j])) if (j == l) break;
     if (i >= j) break;
     exch(a, i, j);
    }
  exch(a, i, r);
  return i;
 }
```

В основата си ефективността на сортирането зависи от това колко добре разделяме масива на части, което от своя страна зависи от стойността на разделящият елемент. Най-добре би било за разделящ елемент да изберем елемент, който би разделил масива приблизително по средата. Един подход е да избегнем най-лошият случай (а именно да изберем за елемент

екстремум за масива) е да използваме произволен елемент от масива за разделящ. По този начин най-лошият случай ще се случва с пренебрежимо малка вероятност. Друг подход е за стойност на разделящият елемент е да се вземе средната стойност на три произволни елемента от масива. По този начин вероятността да вземем осреднена стойност за масива е по-голяма и това би дало подобрения във ефективността на метода за сортиране.
QuickSort е най-бързият универсален алгоритъм за сортиране, въпреки, че в дадени ситуации, той може да бъде значително по-бавен от средната си сложност. В най-добрия случай, при размер на масива N, ако всеки път избираме средният по големина елемент, за цялостното сортиране на масива ще са ни достатъчни lgN разделяния. Тъй като при всяко разделяне ще правим N на брой сравнения, за общия брой сравнения в най-добрият случай получаваме N.lgN. От друга страна обаче в най-лошият случай, при обратно сортиран масив с рамер N и разделящ елемент – екстрамума на масива, сложността на алгоритъма може да стане ‘квадратнична’ N.N. Въпреки всичко средната алгоритмична сложност на QuickSort е Θ(N.lgN), правейки го най-бързият алогиръм за общо ползване известен до момента.